CS 61C Fall 2024

Garcia, Kao Midterm

Solutions last updated: Monday, October 21st, 2024

Print Your Name: _____

Print Your Student ID: _____

You have 110 minutes. There are 7 questions of varying credit (100 points total).

Question:	1	2	3	4	5	6	7	Total
Points:	17	24	10	24	12	13	0	100

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (completely unfilled)
- ● Only one selected option (completely filled)
- ◍ Don't do this (it will be graded as incorrect)

For questions with **square checkboxes**, you may select one or more choices.

- ☐ You can select
- ■ multiple squares
- ■ (completely filled)

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

If an answer requires hex input, you must only use capitalized letters (`0xDEADBEEF` instead of `0xdeadbeef`). For hex and binary, please include prefixes in your answers unless otherwise specified, and do not truncate any leading 0's. For all other bases, do not add any prefixes or suffixes.

Write the statement below in the same handwriting you will use on the rest of the exam.

> I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat; if these answers are not my own work, I may be deducted up to `0x0123 4567 89AB CDEF` points.

_____

_____

_____

_____

_____

Sign your name: _____

Clarifications made during the exam:
Q4: For the RISC-V assembly code in Q4.16-4.19, the addi instructions on lines 3 and 4 should be mv, and the addi instructions on lines 11 and 12 should be add.
Q4: For the previous clarification, Q4.16-4.19 are on page 9, and you should be looking at the codeblock on page 9.

This page intentionally left (mostly) blank.

The exam continues on the next page.

Q1 *Potpourri* **(17 points)**

Q1.1 (1 point) Convert the 8-bit binary `0b0001 1101` to decimal, treating it as an **unsigned** integer.

Solution: 29

Grading: All-or-nothing.

Q1.2 (1 point) For some $n > 0$, an n-bit unsigned integer representation and an n-bit two's complement integer representation can represent the *same number of unique values*.

● True ○ False

For Q1.3-Q1.5, consider an **12-bit** floating point format that follows the IEEE-754 standard, with 1 sign bit, 6 exponent bits (with a standard exponent bias of -31), and 5 mantissa bits.

Q1.3 (2 points) Convert -3.125 into hexadecimal in this floating point format.

0x

Solution: 0xC12

The sign bit should be 1 as we're working with a negative number. Converting 3.125 into binary gives us $0b11.0010 = 0b1.10010 * 2^1$. Since 1 = exponent - bias, our exponent should be 32, which in binary is (`0b100000`). The mantissa would be `0b10010`.

`0x 1 100000 10010 = 0xC12`

Q1.4 (2 points) What is the largest positive denormalized float this format can support, in hexadecimal?

0x

Solution: 0x01F

Denormalized numbers require the exponent bits to all be 0. To maximize the number, the mantissa bits can all be 1. Lastly, the sign bit must be 0 for the number to be positive. Converting the resulting binary number to hexadecimal yields **0x01F**.

Q1.5 (3 points) What is the largest value of n such that the following statement is true?
"Every n-bit two's complement number can be represented by this floating point format."

Solution: `7`

To answer this question, it may be helpful to pose a different, but similar, question we have more experience solving. In particular, from Homework 3, what is the smallest positive integer that our floating point system can not represent?

Recall that the system's mantissa is the biggest limitation to precision. Given a 5-bit mantissa, integers up to 2**6 can be consecutively represented. However, 2**6 + 1 can not be represented as a sixth mantissa bit would be required (`0b1.000001 * 2**6 = 65`).

The range of possible consecutive integers for this floating point system is then [-64, 64] because the sign bit can be either 1 or 0. The largest n for which this statement is true is, therefore, 7 because a 7-bit two's complement number can represent [-64, 63].

Q1.6 (3 points) Convert the following RISC-V machine code into its corresponding instruction. If there is an immediate value, express it in decimal form. Provide the appropriate register names, not numbers, where necessary (i.e. `s5` instead of `x21`).

`0xFFE4 3493`

Solution: sltiu s1 s0 -2

Q1.7 (3 points) Translate the `beq` instruction on line 2 to machine code.

```
1 target: addi t1 x0 -10
2 beq t1 t2 target
```

0x

Solution: 0xFE730EE3

Q1.8 (1 point) The output of the linker may contain pseudoinstructions.

○ True ● False

Grading: All-or-nothing.

Q1.9 (1 point) The compiler outputs an optimized C file.

○ True ● False

Grading: All-or-nothing.

Q2 *61Cafe ☕ (C memory)* **(24 points)**

61C wants to open a 61Cafe and get 61Caffeinated!

Consider the following struct definition:

```
1 typedef struct {
2   char **items;
3   uint32_t num_items;
4 } order_t;
```

Q2.1 (1 point) What is `sizeof(order_t)` on a 32-bit system?

_____ bytes

Solution: 8 bytes.

`items` is 4 bytes, and `num_items` is 4 bytes.

For Q2.2-Q2.4, choose which section of memory the given symbol would live in.

```
1 int main() {
2   order_t order;
3   order.items = malloc(12);
4   return 0;
5 }
```

Q2.2 (1 point) `main`

● Code ○ Static ○ Heap ○ Stack

Q2.3 (1 point) `order`

○ Code ○ Static ○ Heap ● Stack

Q2.4 (1 point) `order.items[1]`

○ Code ○ Static ● Heap ○ Stack

`create_order`: Initializes an `order_t` with the given items.		
Arguments	`char *str`	A space-separated string of items. You may assume that there is a space after the last item in `str`. For example: `"coffee latte mocha espresso "` is 4 items long.
Return value	`order_t *`	A pointer to an `order_t` with the given items.

(17 points) Implement `create_order` to match the described behavior.

```
1  order_t *create_order(char *str) {
2    order_t *ret = calloc(1, sizeof(order_t));
                    Q2.5
3    int num = 0; // Tracks the number of items in the order.
4    while (*str) {
5      int len = 0;
6      while (str[len] != ' ') {
                  Q2.6
7        len++;
8      }
9      ret->items = realloc(ret->items, (num + 1) * sizeof(char*));
                                        Q2.7
10     ret->items[num] = malloc((len + 1) * sizeof(char));
                                      Q2.8
11     memcpy(ret->items[num], str , len );
                  Q2.9          Q2.10  Q2.11
12     ret->items[num][len] = '\0' ;
                              Q2.12
13     str += len + 1;
              Q2.13
14     num++;
15   }
16   ret->num_items = num;
17   return ret;
18 }
```

Solution: Note: If you used `calloc(len + 1, sizeof(char)` and left the `'0'` character blank, we'll give points since this technically works.

Useful C stdlib function prototypes:

```
1 void *malloc(size_t size);
2 void *calloc(size_t num_elements, size_t size);
3 void *realloc(void *ptr, size_t size);
4 void free(void *ptr);
5 void *memcpy(void *dest, void *src, size_t n);
```

Now, consider the `cafe_t` struct below. (`order_t` from earlier is repeated for your convenience.)

```
1 typedef struct {
2   char **items;
3   uint32_t num_items;
4 } order_t;
5
6 typedef struct {
7   order_t *orders;
8   uint32_t num_orders;
9 } cafe_t;
```

`free_cafe`: Frees a `cafe_t`.		
Arguments	`cafe_t *cafe`	A pointer to a `cafe_t` struct to free. You may assume that `orders` is allocated on the heap, and each `order_t` is created via the `create_order` function.
Return value	`void`	

Consider this code for freeing a `cafe_t` struct:

```
1 void free_cafe(cafe_t *cafe) {
2   for (int i = 0; i < cafe->num_orders; i++) {
3     for (int j = 0; j < cafe->orders[i].num_items; j++) {
4       free(/* Answer to Q2.13 */);
5     }
6   }
7   free(cafe->orders);
8   free(cafe);
9 }
```

Q2.14 (1 point) What goes in the blank?

- ○ `cafe.orders[i].items[j]`
- ○ `cafe.orders[i]->items[j]`
- ○ `cafe->orders[i]->items[j]`
- ● `cafe->orders[i].items[j]`

Q2.15 (2 points) The code above is missing one line. Fill in the body of the missing call to free.

```
free(cafe->orders[i].items                                    )
```

Q3 `61('B'++)` *(C Bitwise)* **(10 points)**

An *odd word* is a word where every letter that appears in the word appears an odd number of times.

Here are some example words:

Odd words	Not odd words
`car`	`aa`
`severe`	`avail`
`""` (empty string)	`teases`
`asfjckl`	`addi`

Implement `is_odd_word`, a C function, as follows:

`is_odd_word`: Checks if every letter in `str` appears an odd number of times.		
Arguments	`char *str`	Input word.
Return value	`bool`	Return true if the input is an odd word, and false otherwise.

You may assume that `str` consists only of lowercase letters (ASCII values `0x61` to `0x7A`), and is properly null-terminated.

Hint: Bitwise operations can be used to treat integers as if they were Boolean arrays.

```
1  bool is_odd_word(char *str) {
2    uint32_t a = 0;
3    uint32_t b = 0;
4    while(*str) {                    // Q3.1
5      a ^= 1 << (*str - 0x61);       // Q3.2
6      b |= 1 << (*str - 0x61);       // Q3.3
7      str++;
8    }
9    return a == b;                   // Q3.4
10 }
```

Q4 *RISCy Memory (RISCV Coding)* **(24 points)**

`memcpy`: Copies `n` consecutive bytes from `src` to `dest`.		
Arguments	`a0`	`dest`, a pointer to a block of more than `n` bytes.
	`a1`	`src`, a pointer to a block of more than `n` bytes.
	`a2`	`n`, the number of bytes to copy.

(18 points) Implement `memcpy` below. Assume that the `src` and `dest` blocks of memory do not overlap. For full credit, your solution must use as few store and load operations as possible.

```
1  memcpy:
2      addi t1 x0 0
3      addi t2 a2 -3
4  memcpy_loop: # Copies full words from src to dest.
5      beq t1 a2 memcpy_done
                 Q4.1
6      bge t1 t2 memcpy_tail
7       lw  t3 0( a1 )
          Q4.2      Q4.3
8       sw  t3 0( a0 )
          Q4.4      Q4.5
9      addi a0 a0 4
                  Q4.6
10     addi a1 a1 4
                  Q4.7
11     addi t1 t1 4
                  Q4.8
12     j memcpy_loop
13 memcpy_tail: # Copies remaining data (if any) from src to dest.
14      lb  t3 0( a1 )
          Q4.9      Q4.10
15      sb  t3 0( a0 )
          Q4.11     Q4.12
16     addi a0 a0 1
                  Q4.13
17     addi a1 a1 1
                  Q4.14
18     addi t1 t1 1
19     blt t1 a2 memcpy_tail
                 Q4.15
20 memcpy_done: # Has no return value
21     jr ra
```

Now, consider the mystery functions `bar` and `foo` which both take in two arguments `a0` and `a1`, and return one output `a0`. `foo` follows calling convention but `bar` (shown below) does not.

```
1  bar:
2    # Q4.16 Prologue
3    addi s0 a0
4    addi s1 a1
5    addi t0 a1 4
6
7    # Q4.17 Save
8    jal foo
9    # Q4.17 Load
10
11   addi a0 a0 t0
12   addi a0 a0 a1
13
14   # Q4.16 Epilogue
15   jr ra
```

For Q4.16-Q4.19, select the minimum number of registers that need to be saved and restored for the function to follow standard RISC-V calling convention.

Q4.16 (2 points) Select all registers we need to save to and restore from the stack at the locations marked Q4.16.

- ☐ `a0`
- ☐ `a1`
- ☒ `s0`
- ☒ `s1`
- ☐ `t0`
- ○ None

Solution: Registers `s0` and `s1` are callee-saved so their values must persist after the function call. Because their values are modified on lines 3 and 4, their previous values must be saved and restored in the prologue and epilogue.

Q4.17 (2 points) Select all registers we need to save to and restore from the stack at the locations marked Q4.17.

- ☐ `a0`
- ☒ `a1`
- ☐ `s0`
- ☐ `s1`
- ☒ `t0`
- ○ None

Solution: Registers `a1` and `t0` are caller-saved so their values may change during the jump to `foo` and must be saved to be used in lines 11 and 12. Note that `a0` is the return value of `foo` so we do not need to save its value before jumping to `foo`.

Q4.18 (1 point) Does `bar` need to save `ra`?

- (●) Yes
- ( ) No
- ( ) Depends on the implementation of `foo`

Solution: `ra` is caller-saved and must be restored if modified. Note that `jal` stores a new value into `ra`.

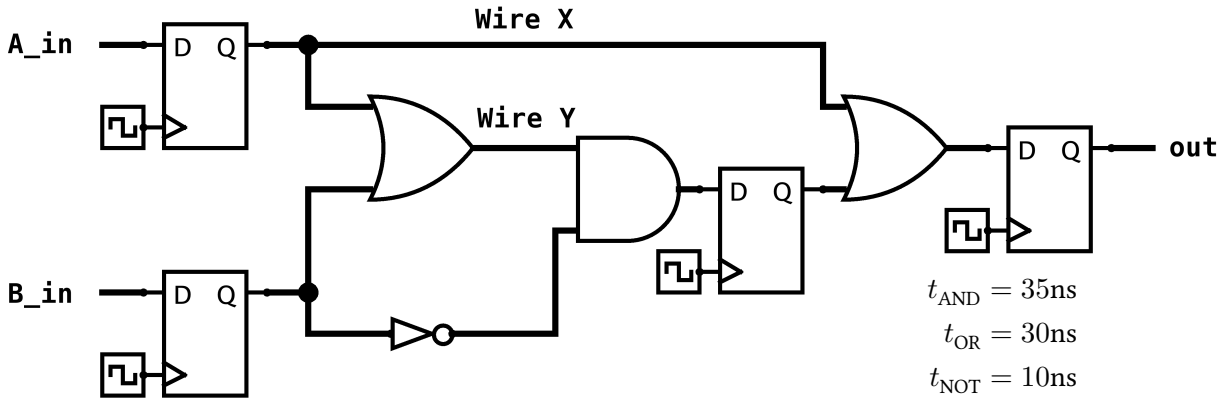
Q4.19 (1 point) Does `foo` need to save `ra`?

- ( ) Yes
- ( ) No
- (●) Depends on the implementation of `foo`

Solution: `foo` only needs to save `ra` if the implementation of `foo` modifies the value of `ra`.

Q5 *SDS* **(12 points)**

Consider the following circuit. Assume all registers are connected to the same clock and `A_in`, `B_in`, and `out` will not cause any hold time violations.



Q5.1 (2 points) What is the **minimum clock period** for this circuit to function properly, in nanoseconds?

ns

Solution: 87ns. The longest combinational logic path goes from the top left register, through the left OR gate, through the AND gate, and into the middle register, giving a total of 65ns. The minimum allowable clock period = $t_{\text{clk-to-q}} + t_{\text{setup}} + t_{\text{longest CL path}}$. This gives 12ns + 10ns + 65ns = 87ns.

Q5.2 (2 points) What is the **maximum hold time** the registers can have such that there are no hold time violations, in nanoseconds?

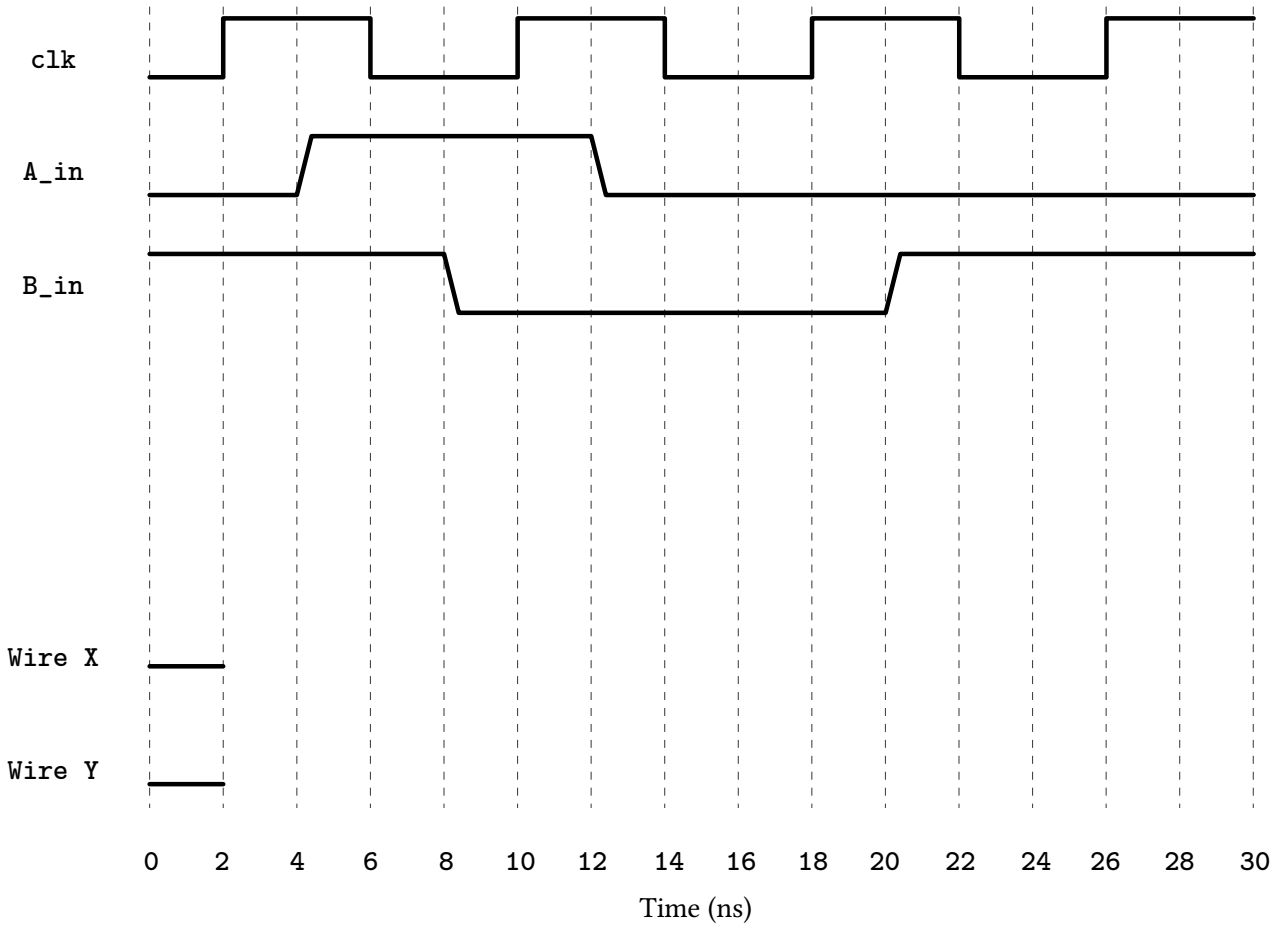
ns

Solution: 42ns. The shortest combinational logic path goes from the top left register, through the right OR gate, and into the rightmost register. The maximum hold time = $t_{\text{clk-to-q}} + t_{\text{shortest CL path}}$. This gives 12ns + 30ns = 42ns.

Now, consider the same circuit with the modified timings and the given input signals (`A_in` and `B_in`).

$$t_{\text{AND}} = t_{\text{OR}} = t_{\text{NOT}} = 1\text{ns}, \;\; t_{\text{clk-to-q}} = 2\text{ns}$$

You may assume that setup and hold times are negligible. At time $t = 0$, all registers output value 0. Use the waveform diagram as scratch space, but your waveforms **will not** be graded.



For Q5.3-Q5.4, write all times that the wire changes value between $t = 2$ns and $t = 30$ns (inclusive). Format your answer as a comma-separated list of the times, in ascending order (e.g. 2, 8, 20, 26 ns).

Q5.3 (4 points) Wire X changes value at:

12, 20	ns

Q5.4 (4 points) Wire Y changes value at:

5, 21, 29	ns

Q6 *Boolean Logic & FSM* **(13 points)**

For Q6.1-Q6.2, write a Boolean expression for the given truth table. For full credit, you must use at most the number of operators given. NOT (`!`), AND (`*`), OR (`+`) each count as one operator. We will assume standard C operator precedence, so use parentheses when uncertain.

Your answer may consist of the following characters:

Inputs	NOT	AND	OR	Constants	Parentheses
`A`, `B`, `C`	`!`	`*`	`+`	`0`, `1`	`()`

Q6.1 (2 points) 3 operators

A	B	C	Out
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Solution: A + !(B + C)

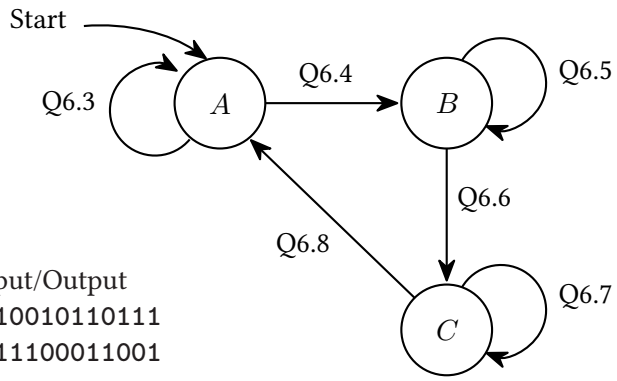
Q6.2 (2 points) 3 operators

A	B	C	Out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Solution: B * (!A + C)

The following FSM outputs `1` if the number of `1`'s seen so far is a multiple of `3`. Otherwise, it should output `0`.

Reminder: `0` is a multiple of `3`.



Example Input/Output
Input: `01110010110111`
Output: `10011100011001`

For each of the numbered arrows, mark the correct FSM state transition.

Q6.3 (1.5 points)	○ `0/0`	● `0/1`	○ `1/0`	○ `1/1`
Q6.4 (1.5 points)	○ `0/0`	○ `0/1`	● `1/0`	○ `1/1`
Q6.5 (1.5 points)	● `0/0`	○ `0/1`	○ `1/0`	○ `1/1`
Q6.6 (1.5 points)	○ `0/0`	○ `0/1`	● `1/0`	○ `1/1`
Q6.7 (1.5 points)	● `0/0`	○ `0/1`	○ `1/0`	○ `1/1`
Q6.8 (1.5 points)	○ `0/0`	○ `0/1`	○ `1/0`	● `1/1`

Q7 *Coloring Book* **(0 points)**

These questions will not be assigned credit; feel free to leave them blank.

Q7.1 (0 points) `#CCFF00` or `#020F61`?

Solution: ...

Q7.2 (0 points) Green or Purple?

Solution: There is only one correct answer...

Q7.3 (0 points) RISC or CISC?

Solution: We love RISC!

Q7.4 (0 points) If there's anything else you want us to know, or you feel like there was an ambiguity in the exam, please put it in the box below.

For ambiguities, you must qualify your answer and provide an answer for both interpretations. For example, "if the question is asking about A, then my answer is X, but if the question is asking about B, then my answer is Y". You will only receive credit if it is a genuine ambiguity and both of your answers are correct. We will only look at ambiguities if you request a regrade.

Solution: ¯_(ツ)_/¯